

Generación de animación 2D de un personaje articulado mediante redes neuronales recurrentes sobre un estado cinemático

Jorge Andrés Zúñiga Delgado
Maestría en Inteligencia Artificial — Aprendizaje Profundo
Proyecto Final

24 de julio de 2026

Resumen

Presentamos un sistema de síntesis de movimiento para un personaje 2D (un ninja con katana) que separa de forma estricta la representación del movimiento y su visualización. El personaje se describe mediante un **estado cinemático** de 17 variables: la posición y orientación de la cadera, sus velocidades, doce ángulos articulares relativos y la fase del movimiento. Las coordenadas de las articulaciones nunca se almacenan, sino que se derivan por **cinemática directa**. Sobre esta representación construimos un conjunto de datos de $600 \times 40 \times 17$ generado de forma procedural y entrenamos redes recurrentes (RNN, LSTM y GRU) para predecir el siguiente estado a partir de una ventana temporal. El mejor modelo alcanza un error cuadrático medio de validación de 0,0071 en el espacio normalizado, con errores de $MAE = 1,42$ y $RMSE = 2,18$ en unidades físicas, y genera animaciones autoregresivas visualmente correctas para los diez movimientos considerados. Una extensión condicionada por acción permite además encadenar coreografías a demanda. Puesto que la salida de la red reutiliza el mismo motor de cinemática y de render de la etapa procedural, convertir esa salida en imagen no exigió modificar el motor gráfico.

1. Introducción

La síntesis automática de movimiento es un problema central en gráficos por computadora y videojuegos. El enfoque clásico, animar a mano fotograma a fotograma o capturar movimiento real, resulta costoso y poco flexible. Los métodos basados en aprendizaje profundo prometen generar movimiento continuo, variado y controlable a partir de datos.

Una decisión determinante es *qué* debe predecir la red. Si el modelo estima directamente las coordenadas (x, y) de cada articulación, nada garantiza que los huesos conserven su longitud: pequeños errores producen extremidades que se estiran o se encogen, un artefacto visualmente inaceptable. Por ese motivo adoptamos una representación angular y jerárquica, en la que la red predice ángulos relativos entre articulaciones y la

geometría se reconstruye después por cinemática directa. Así, la invariancia de las longitudes óseas deja de ser algo que el modelo debe aprender y pasa a estar garantizado por construcción.

Las contribuciones de este trabajo son cuatro: un motor de animación esquelética 2D con cinemática directa, límites articulares y renderizado por piezas; un generador procedural de diez movimientos que actúa como fuente de datos etiquetados y reproducibles; el diseño y la evaluación comparada de modelos recurrentes sobre el estado cinemático; y una variante condicionada por acción capaz de encadenar varios movimientos consecutivos.

2. Trabajos relacionados

El modelado de secuencias con redes recurrentes se apoya en las celdas con compuertas: la LSTM [1] y, posteriormente, la GRU [2], que logra un desempeño comparable con menos parámetros. Ambas constituyen las arquitecturas que comparamos en este trabajo.

En el dominio específico del movimiento, Martínez *et al.* [3] estudiaron la predicción de movimiento humano con redes recurrentes y mostraron que las arquitecturas simples con conexiones residuales son sorprendentemente competitivas. Su aportación más relevante para nosotros es el diagnóstico de la *deriva* de estos modelos en horizontes largos, un fenómeno que observamos y discutimos en la Sección 6.

Dos trabajos justifican decisiones concretas de nuestro diseño. Las *Phase-Functioned Neural Networks* de Holden *et al.* [4] demostraron que una variable de **fase**, que indica en qué punto del ciclo se encuentra la acción, es determinante para lograr locomoción estable; de ahí que incluyamos la fase dentro del propio estado. Por su parte, Villegas *et al.* [5] incorporaron una capa de cinemática directa dentro de la red para garantizar huesos de longitud constante, idea que nosotros aplicamos situando la cinemática directa fuera del modelo, como paso de decodificación.

A diferencia de esos trabajos, nuestro escenario es 2D y controlado: el generador procedural provee datos ilimitados y sin ruido, lo que aísla el efecto de la arquitectura

del de la calidad de los datos.

3. Representación cinemática y renderizado

3.1. Esqueleto y cinemática directa

El personaje es un árbol jerárquico de 17 nodos cuya raíz es la cadera. De la cadera nacen el tronco, y de éste el cuello, la cabeza y ambos brazos, así como las dos piernas; la katana cuelga de la mano derecha mediante una articulación adicional. Cada hueso posee una longitud constante ℓ_j , un ángulo de reposo β_j relativo a su padre y, cuando es articulado, una variable θ_j . La posición de cada nodo se obtiene de forma recursiva:

$$\phi_j = \phi_{\text{padre}(j)} + \beta_j + s_j \theta_j, \quad (1)$$

$$\mathbf{p}_j = \mathbf{p}_{\text{padre}(j)} + \ell_j (\cos \phi_j, \sin \phi_j), \quad (2)$$

donde ϕ_j es el ángulo absoluto del hueso y $s_j \in \{-1, +1\}$ codifica el sentido anatómico de la flexión, pues el codo flexiona hacia delante y la rodilla hacia atrás. La raíz parte de $(\text{root_x}, \text{root_y})$ con orientación $\text{root_rotation} + 90^\circ$.

Cada articulación tiene además **límites físicos** que se aplican por saturación antes de propagar la cadena: codo $[0^\circ, 150^\circ]$, rodilla $[0^\circ, 140^\circ]$, cuello $[-40^\circ, 40^\circ]$ y espalda $[-30^\circ, 30^\circ]$. Esto impide poses anatómicamente imposibles incluso si el modelo predice valores fuera de rango y, como veremos, estabiliza la generación.

3.2. Del estado al personaje: el proceso de render

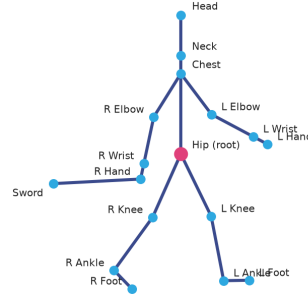
El render cierra el ciclo de validación: sólo viendo el movimiento se distingue un error irrelevante de otro que arruina la pose. El proceso es idéntico para el generador procedural y para la red (Fig. 1): se recorre el árbol con las ecuaciones (1) y (2) para obtener el ángulo y la posición de los 17 nodos; a cada hueso le corresponde una imagen PNG (generada también de forma procedural) que se rota ϕ_j grados alrededor de su pivote y se ancla sobre la articulación padre, de modo que las piezas nunca se separan; finalmente se componen de atrás hacia delante. Lo esencial es que el motor de render desconoce si el estado procede de una fórmula o de una red, por lo que sustituir el generador por el modelo entrenado no obligó a modificarlo.

4. Construcción del conjunto de datos

4.1. Cómo se generan las secuencias

Los datos no proceden de captura de movimiento sino de un **generador procedural** propio. Para cada movi-

(a) Esqueleto jerárquico + cinemática directa



(b) Render con piezas PNG rotadas



Figura 1: (a) Árbol jerárquico y coordenadas obtenidas por cinemática directa. (b) Render final componiendo las piezas PNG rotadas sobre sus pivotes.

miento definimos los ángulos articulares como funciones continuas del tiempo normalizado $t \in [0, 1]$, construidas con senos, cosenos, interpolaciones suaves y curvas de campana. La continuidad de estas funciones es precisamente lo que hace que la animación resultante se perciba fluida al reproducirse a 30 fotogramas por segundo.

Conviene verlo en un caso concreto. La patada se modela con una curva de campana centrada en el instante del impacto,

$$e(t) = \exp\left(-\frac{(t - t_g)^2}{2\sigma^2}\right), \quad (3)$$

que vale cero en los extremos y alcanza su máximo en el golpe. Los ángulos de la pierna derecha interpolan entre reposo e impacto con ese factor:

$$\text{right_hip}(t) = \text{lerp}(-10^\circ, r, e(t)), \quad (4)$$

$$\text{right_knee}(t) = \text{lerp}(70^\circ, 8^\circ, e(t)), \quad (5)$$

de modo que la cadera eleva la pierna al frente mientras la rodilla se extiende, y ambas regresan a la posición inicial.

Un aspecto esencial es que los parámetros se **sortean al azar en cada ejecución**: en la patada varían el instante del golpe $t_g \sim U(0,45, 0,60)$, la velocidad del ataque (que fija el ancho σ) y el alcance $r \sim U(85^\circ, 105^\circ)$; en la marcha, la longitud del paso, la amplitud de los brazos y la inclinación del torso; en el salto, su altura. Así nunca se generan dos secuencias idénticas, lo que impide memorizar trayectorias y actúa como una forma intensiva de aumento de datos.

4.2. Estructura del tensor

Generamos 60 variantes de cada uno de los 10 movimientos, fijando la duración en 40 fotogramas para obtener un tensor rectangular. El resultado tiene forma

Cuadro 1: Significado de las $D = 17$ columnas del vector de estado.

#	Variable	Unidad	Significado
0	root_x	px	traslación horizontal de la cadera
1	root_y	px	altura de la cadera (saltos)
2	root_rotation	°	orientación global del cuerpo
3	velocity_x	px/f	velocidad horizontal de la raíz
4	velocity_y	px/f	velocidad vertical de la raíz
5	torso_angle	°	flexión de la espalda [-30, 30]
6	neck_angle	°	cuello [-40, 40]
7	left_shoulder	°	hombro izq. [-170, 170]
8	left_elbow	°	codo izq. [0, 150]
9	right_shoulder	°	hombro der. [-170, 170]
10	right_elbow	°	codo der. [0, 150]
11	left_hip	°	cadera izq. [-120, 120]
12	left_knee	°	rodilla izq. [0, 140]
13	right_hip	°	cadera der. [-120, 120]
14	right_knee	°	rodilla der. [0, 140]
15	sword_angle	°	katana respecto de la mano
16	phase	—	progreso normalizado [0, 1)

$(N, T, D) = (600, 40, 17)$, y conviene interpretar cada eje por separado.

El primer eje recorre las **600 secuencias**, cada una de las cuales es una ejecución completa de una acción. Están ordenadas por movimiento, de modo que las secuencias 0 a 59 son variantes de *idle*, las 60 a 119 de *walk*, y así sucesivamente. El segundo eje recorre los **40 fotogramas** de esa ejecución: cada posición es un instante de tiempo. El tercer eje recorre las **17 variables** del estado, descritas en la Tabla 1.

Un ejemplo concreto aclara la lectura. La secuencia 430 pertenece al grupo de las patadas, que ocupa los índices 420 a 479. El elemento `data[430, 22, 13]` contiene el ángulo de la cadera derecha (variable 13) en el fotograma 22 de esa ejecución, y su valor real es $103,9^\circ$, que corresponde al instante de máxima elevación de la pierna. En ese mismo fotograma, `data[430, 22, 14]` vale $8,3^\circ$, es decir, la rodilla está prácticamente extendida: exactamente la mecánica del impacto descrita más arriba.

Las rebanadas del tensor tienen una interpretación igualmente clara. `data[430, 22, :]` es un vector de 17 números que constituye la *pose completa* del personaje en ese instante, la fotografía instantánea de todo el cuerpo. En cambio, `data[430, :, 13]` es la *trayectoria temporal* de esa única articulación a lo largo de toda la patada: una curva ascendente y descendente que reproduce la mecánica del golpe.

Merece subrayarse que una fila no contiene un ángulo aislado, sino los doce ángulos articulares de forma simultánea junto con la raíz, sus velocidades y la fase. Una acción es, por tanto, una trayectoria suave de poses en R^{17} .

La inclusión de las velocidades y de la fase no es accesorio. Las velocidades, calculadas como diferencias finitas de la raíz, aportan información dinámica de primer orden que de otro modo la red tendría que inferir. La fase, inspirada en [4], indica en qué punto del ciclo se encuentra la acción y ayuda a desambiguar poses visual-

mente parecidas pero temporalmente opuestas, como la pierna subiendo frente a la pierna bajando.

4.3. Ventanas temporales y normalización

Convertimos las secuencias en pares supervisados mediante una **ventana deslizante**: las poses de las posiciones i a $i + W - 1$ se usan para predecir la pose $i + W$, con $W = 16$. La partición entre entrenamiento y validación se realiza **por secuencias** y no por ventanas, en proporción 85/15, para evitar que fragmentos de una misma ejecución aparezcan en ambos conjuntos. Se obtienen así 12 240 ventanas de entrenamiento, con $X \in R^{12240 \times 16 \times 17}$, y 2 160 de validación.

Como las variables tienen escalas muy dispares, pues conviven píxeles y grados, se estandarizan mediante $\tilde{x} = (x - \mu)/\sigma$ con estadísticos calculados únicamente sobre el conjunto de entrenamiento. El normalizador se guarda en disco porque resulta imprescindible durante la inferencia: la red opera en el espacio normalizado y su salida debe deshacerse esa transformación antes de renderizar.

5. Arquitectura del modelo

El problema se formula como una regresión autoregresiva: dada la ventana $\mathbf{X} = (\tilde{\mathbf{s}}_i, \dots, \tilde{\mathbf{s}}_{i+W-1})$, se debe predecir $\tilde{\mathbf{s}}_{i+W}$. La red consta de cuatro elementos encadenados. Primero, una pila de dos capas recurrentes con 64 unidades ocultas y *dropout* de 0,1 entre ellas, que procesa los 16 fotogramas de la ventana. Segundo, una extracción del vector correspondiente al **último paso temporal** de la salida recurrente, que resume el contexto de toda la ventana. Tercero, una capa densa de 64 unidades con activación ReLU. Y cuarto, una capa densa final de 17 salidas, una por cada variable del estado.

El tipo de celda recurrente es un único parámetro configurable, lo que permite comparar RNN simple, LSTM y GRU manteniendo idéntico el resto del sistema. En su versión GRU el modelo tiene 46 161 parámetros entrenables, una cifra modesta que confirma que la dificultad del problema reside en la representación y no en la capacidad del modelo.

El entrenamiento minimiza el error cuadrático medio sobre el espacio normalizado, donde todas las variables pesan por igual, mediante el optimizador Adam con tasa de aprendizaje 10^{-3} , lotes de 256 muestras y 15 épocas, conservando los pesos que producen la menor pérdida de validación.

Variante condicionada por acción. El modelo anterior continúa el movimiento que observa, pero no permite *solicitar* una acción concreta. Para encadenar coreografías añadimos a la entrada de cada paso tem-

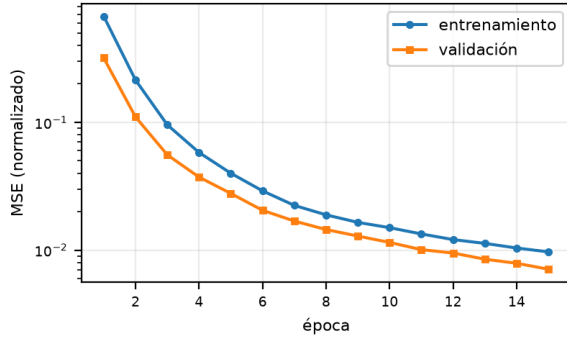


Figura 2: Curvas de pérdida (error cuadrático medio normalizado, escala logarítmica) del modelo GRU.

poral un vector *one-hot* que indica la acción deseada, de manera que la dimensión de entrada pasa de $17 + 10 = 27$. Este modelo se entrena sobre secuencias compuestas, esto es, concatenaciones de dos o tres movimientos con transiciones interpoladas, para que aprenda también los tránsitos entre acciones. Se emplearon 12 108 ventanas y una GRU de 96 unidades durante 16 épocas, alcanzando una pérdida final de 0,0162.

6. Resultados y discusión

6.1. Convergencia y error

La Fig. 2 muestra las curvas de pérdida del modelo GRU. El error de validación desciende de 0,3187 a 0,0071 a lo largo de 15 épocas de forma monótona y sin separarse de la curva de entrenamiento, lo que indica ausencia de sobreajuste. Este comportamiento era previsible, ya que el generador provee variabilidad ilimitada y opera como una forma intensiva de aumento de datos.

Sobre las 2160 ventanas de validación, el error de predicción a un paso expresado en unidades físicas es de $MAE = 1,42$ y $RMSE = 2,18$. Dado que los ángulos recorren rangos de hasta 150° , un error medio próximo a $1,4^\circ$ resulta visualmente imperceptible.

6.2. Comparación de arquitecturas

La Tabla 2 compara las tres celdas en igualdad de condiciones, con 10 épocas cada una. Las celdas con compuertas superan claramente a la RNN simple en todas las métricas de error. Entre ellas, LSTM y GRU quedan muy próximas, con un MAE de 1,73 frente a 1,78, pero la GRU entrena aproximadamente un 15 % más rápido por disponer de menos parámetros por celda. La RNN simple es la más veloz y a la vez la menos precisa, y cualitativamente es también la que antes se degrada en generación larga. Este patrón coincide con lo reportado en predicción de movimiento humano [3].

Cuadro 2: Comparación de arquitecturas (10 épocas, mismo conjunto y partición).

Modelo	t_{entren} (s)	t_{infer} (s)	MSE val.	MAE	RMSE
RNN	2,11	0,0039	0,0137	1,929	3,577
LSTM	6,65	0,0075	0,0118	1,727	3,107
GRU	5,64	0,0073	0,0122	1,778	3,303

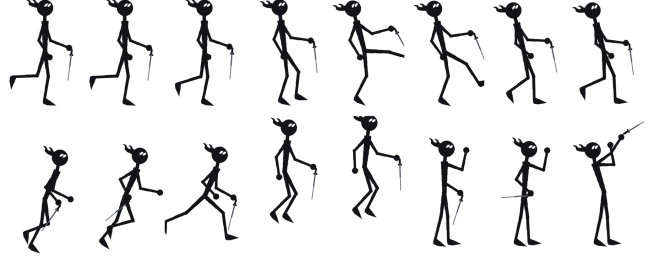


Figura 3: (a) Patada generada autoregresivamente por la GRU. (b) Secuencia encadenada en la que el ninja corre, salta y ataca, con transiciones suaves.

6.3. Evaluación cualitativa

La evaluación visual constituye la prueba definitiva. Partiendo de una semilla de 16 fotogramas reales, la red genera de forma autoregresiva los 24 restantes para los diez movimientos, y todos resultan reconocibles y anatómicamente plausibles. La Fig. 3(a) muestra una patada generada por el modelo en la que la cadera se eleva y la rodilla se extiende en el instante del impacto, reproduciendo la mecánica del golpe. La Fig. 3(b) muestra una secuencia encadenada de 124 fotogramas en la que el personaje corre, salta y ataca de forma continua.

6.4. Discusión y limitaciones

Observamos dos fenómenos relevantes. El primero es que la saturación a los límites articulares actúa como regularizador durante la inferencia: al reinyectar en la ventana una predicción ya saturada, se evita que pequeños errores se acumulen hasta producir poses imposibles. El segundo es que la estabilidad depende de la acción. Los movimientos en contacto con el suelo, como correr, patear o cortar, se sostienen bien, mientras que el salto, que incluye una fase aérea donde la raíz sigue una parábola, tiende a degradarse en horizontes largos. Este comportamiento es consistente con la deriva descrita por Martinez *et al.* [3] y constituye la principal limitación del sistema. Su origen está en el desajuste entre el entrenamiento, que emplea siempre valores reales como entrada, y la inferencia, que realimenta las propias predicciones del modelo; el *scheduled sampling* [6] es el remedio estándar para este problema.

7. Salidas del modelo: interpretación y ejecución

Cada invocación devuelve un vector de 17 números normalizados: la pose siguiente, no una imagen ni una clase. Se interpreta deshaciendo la normalización ($x = \tilde{x}\sigma + \mu$) y leyendo cada componente según la Tabla 1. La animación completa se obtiene de forma autoregresiva: se parte de una ventana semilla, se predice la pose siguiente, se añade a la ventana y se descarta la más antigua; cada pose se satura a los límites articulares, se convierte en coordenadas por cinemática directa y se dibuja. El visualizador se lanza con `python main.py` y el cuaderno `development.ipynb` reproduce dataset, entrenamiento, comparación y generación; los artefactos `best_model.pt` y `normalizer.npz` permiten inferir sin reentrenar. La implementación emplea PyTorch.

8. Conclusiones

Hemos mostrado que una red recurrente compacta, de unos 46 mil parámetros, es capaz de aprender y regenerar diez movimientos acrobáticos de un personaje articulado cuando el problema se plantea sobre un estado cinemático angular en lugar de sobre coordenadas. La cinemática directa garantiza por construcción que las longitudes de los huesos se conserven, y los límites articulares aportan una regularización física que estabiliza la generación autoregresiva.

Las celdas con compuertas superan a la RNN simple y resultan equivalentes entre sí, con ventaja de coste para la GRU; las diez acciones son reconocibles y el condicionamiento por acción permite encadenar coreografías a demanda.

Las limitaciones principales son la degradación del salto en horizontes largos, la predicción de la posición absoluta de la raíz, que acumula error, y el carácter sintético de los datos, por lo que la generalización a captura real queda sin verificar. Como trabajo futuro proponemos entrenar con *scheduled sampling* [6], predecir incrementos del estado en lugar de valores absolutos reconstruyendo la raíz por integración de la velocidad, modular los pesos con la fase siguiendo a [4], y validar el procedimiento sobre datos reales de captura de movimiento.

Referencias

- [1] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [2] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” in *EMNLP*, 2014.
- [3] J. Martinez, M. J. Black, and J. Romero, “On Human Motion Prediction Using Recurrent Neural Networks,” in *CVPR*, 2017.
- [4] D. Holden, T. Komura, and J. Saito, “Phase-Functioned Neural Networks for Character Control,” *ACM Trans. Graph. (SIGGRAPH)*, vol. 36, no. 4, 2017.
- [5] R. Villegas, J. Yang, D. Ceylan, and H. Lee, “Neural Kinematic Networks for Unsupervised Motion Retargeting,” in *CVPR*, 2018.
- [6] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, “Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks,” in *NeurIPS*, 2015.